

```
newcabinet = merging(left,right)
return(newcabinet)
```

[Listing 4-5: Implementing merge sort with recursion](#)

You can see that this function is nearly identical to our `mergesort_four_elements()` to function. The crucial difference is that to create the sorted left and right cabinets, it doesn't call another function that works on smaller lists. Rather, it calls itself on the smaller list ❶ ❷. Merge sort is a *divide and conquer* algorithm. We start with a large, unsorted list. Then we split that list repeatedly into smaller and smaller chunks (the dividing) until we end up with sorted (conquered) one-item lists, and then we simply merge them back together successively until we have built back up to one big sorted list. We can call this merge sort function on a list of any size and check that it works:

```
cabinet = [4,1,3,2,6,3,18,2,9,7,3,1,2.5,-9]
newcabinet = mergesort(cabinet)
print(newcabinet)
```

When we put all of our merge sort code together, we get [Listing 4-6](#).

```
def merging(left,right):
    newcabinet = []
    while(min(len(left),len(right)) > 0):
        if left[0] > right[0]:
            to_insert = right.pop(0)
            newcabinet.append(to_insert)
        elif left[0] <= right[0]:
            to_insert = left.pop(0)
            newcabinet.append(to_insert)
    if(len(left) > 0):
        for i in left:
            newcabinet.append(i)
    if(len(right) > 0):
        for i in right:
            newcabinet.append(i)
    return(newcabinet)

import math

def mergesort(cabinet):
```